

**SCHOOL OF
COMPUTING
CHENNAI**

**DATA STRUCTURES AND ALGORITHMS LAB
(24CYS283)**

LABORATORY MANUAL

**B. TECH
(IIND YEAR – IVTH SEM)
(2024-2028)**

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING (CYBER SECURITY)**

TABLE OF CONTENTS

Ex No	Name of the Experiments	Page No
1	Implementation of linear sorting, and Divide and Conquer based sorting algorithms	
2	Array and Linked list implementation of List, Stack and Queue ADTs	
3	Doubly linked list and circular linked list	
4	Applications of List, Stack and Queue ADTs	
5	Array based and linked structure-based implementation of Binary Tree operations	
6	Implementation of priority queues-heaps	
7	List based implementation of hash tables	
8	Implementation of skip lists	
9	Implementation of binary search trees, AVL trees and Splay trees	
10	Implementation of graph traversals	

Course Mentor

Program Chair/Chairperson

Experiment: 1

Implementation of linear sorting, and Divide and Conquer based sorting algorithms

Q1: Bucket Sort

Imagine a teacher needs to sort the scores of a recent test, where all scores are between 0 and 100.

Input Data: A list of student scores (e.g., 85, 92, 63, 75, 99, 61, 88).

The Buckets: The teacher decides to create 10 buckets, one for each score decade:

Bucket 0: Scores 0-9

Bucket 1: Scores 10-19

...

Bucket 9: Scores 90-100

The Process is that the teacher quickly places each student's score into the correct bucket based on the tens digit (the distribution phase). For example, 85 and 88 go into Bucket 8. Once all scores are distributed, the teacher sorts the few scores within each individual bucket (the secondary sort phase). For example, in Bucket 8, they sort (85, 88) to become (85, 88). Finally, the teacher collects the scores sequentially from Bucket 0 up to Bucket 9.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class BucketSort {
private:
    vector<int> buckets[10]; // 10 buckets

public:
    void sortScores(int arr[], int n) {

        // Distribution Phase
        for(int i = 0; i < n; i++) {
```

```

        int index = arr[i] / 10; // tens digit
        if(index > 9) index = 9; // for score 100
        buckets[index].push_back(arr[i]);
    }

    // Secondary Sort Phase (sort each bucket)
    for(int i = 0; i < 10; i++) {
        sort(buckets[i].begin(), buckets[i].end());
    }

    // Collect Phase
    int k = 0;
    for(int i = 0; i < 10; i++) {
        for(int j = 0; j < buckets[i].size(); j++) {
            arr[k++] = buckets[i][j];
        }
    }
}

void display(int arr[], int n) {
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

};

int main() {
    int scores[] = {85, 92, 63, 75, 99, 61, 88};
    int n = 7;

    BucketSort obj;

    cout << "Unsorted Scores: ";
    obj.display(scores, n);

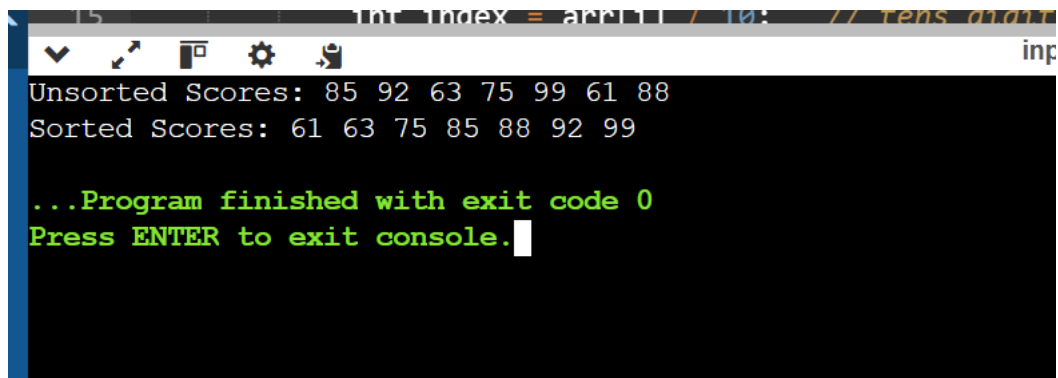
    obj.sortScores(scores, n);

    cout << "\nSorted Scores: ";
    obj.display(scores, n);

    return 0;
}

```

OUTPUT:



```
int index = arr[1] / 10; // tens digit
Unsorted Scores: 85 92 63 75 99 61 88
Sorted Scores: 61 63 75 85 88 92 99

...Program finished with exit code 0
Press ENTER to exit console.
```

Q2: Radix Sort

Imagine you are working with the HR department of a large company that assigns each employee a unique 5-digit Employee ID. The company recently ran an internal report and has a huge, unsorted list of these 5-digit IDs that needs to be sorted numerically for efficient lookup and processing.

Input Data: A large array of positive 5-digit integers (Employee IDs).

Goal: Sort the Employee IDs in ascending order as quickly as possible.

Expected Output:

Unsorted Employee IDs: 90175 12044 33501 86704 21975 40012 65000

Sorted Employee IDs: 12044 21975 33501 40012 65000 86704 90175

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>
using namespace std;

class RadixSort {
private:

    int getMax(int arr[], int n) {
        int max = arr[0];
        for(int i = 1; i < n; i++)
            if(arr[i] > max)
                max = arr[i];
        return max;
    }
}
```

```

void countSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    // Count occurrences
    for(int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Cumulative count
    for(int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build output array
    for(int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy back
    for(int i = 0; i < n; i++)
        arr[i] = output[i];
}

public:
void sortIDs(int arr[], int n) {
    int max = getMax(arr, n);

    for(int exp = 1; max / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

void display(int arr[], int n) {
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

};

int main() {

    int ids[] = {90175, 12044, 33501, 86704, 21975, 40012, 65000};

```

```

int n = 7;

RadixSort obj;

cout << "Unsorted Employee IDs: ";
obj.display(ids, n);

obj.sortIDs(ids, n);

cout << "\nSorted Employee IDs: ";
obj.display(ids, n);

return 0;
}

```

OUTPUT:

```

input
Unsorted Employee IDs: 90175 12044 33501 86704 21975 40012 65000
Sorted Employee IDs: 12044 21975 33501 40012 65000 86704 90175
...Program finished with exit code 0
Press ENTER to exit console.

```

Q3: Count Sort

The Scenario: The "Speed-Run" Leaderboard

You are developing a mini-game where players complete a level as fast as possible. The game assigns a Star Rating (from 0 to 9) based on their performance.

After a local tournament, you have a list of ratings for 6 players: [4, 9, 2, 4, 7, 6]. You need to sort these ratings in ascending order to display them on the scoreboard. Since the range of ratings is very small (0–9), your lead developer insists you use Counting Sort for maximum efficiency.

Implement the above scenario using C++/Python

CODE:

```

#include<iostream>
#include<algorithm>

```

```

#include<vector>

using namespace std;
int main () {
int n,m;
cout <<"enter no of values: ";
cin>>n;
int input[n];
for(int i=0;i<n;i++) {
    cout<<"enter "<<i+1<<" value";
    cin>>m;
    input[i]=m;
}
int mx = *max_element(input,input+n);

int arr[mx] = {0};
for(int i=0;i<n;i++) {
    int a=input[i]-1;
    arr[a]++;
}
for(int i=1;i<mx;i++) {
    arr[i] += arr[i-1];
}

int output[n];
for(int i=0;i<n;i++) {
    int j=input[i];
    int p=arr[j-1];
    output[p-1]=input[i];
    arr[j-1]--;
}
for(int i=0;i<n;i++) {
    cout<<output[i]<<" ";
}
return 0;
}

```

OUTPUT:

```
"C:\Users\ganes\OneDrive\De  X + v
enter no of values: 5
enter 1 value23
enter 2 value12
enter 3 value15
enter 4 value16
enter 5 value90
12 15 16 23 90
Process returned 0 (0x0)    execution time : 16.324 s
Press any key to continue.
|
```

Q4: Quick Sort

The Scenario: Firewall Severity Prioritization

You are debugging a firewall module that processes incoming threat alerts. The system receives a batch of 7 numeric "Severity Codes": {56, 12, 78, 34, 23, 90, 15}

The system uses a Quick Sort algorithm to organize these alerts from lowest to highest priority. The current implementation is hardcoded to always select the first element of any list (or sub-list) as the Pivot.

Task: As the lead engineer, perform a manual "dry run" of the sorting process to verify the logic:

The First Partition: Using 56 as the initial pivot, show how the array is rearranged so that smaller codes are on the left and larger codes are on the right.

Recursive Breakdown: After the first split, identify the Left Sub-array and Right Sub-array. Repeat the partitioning process for these sub-arrays, clearly identifying the new pivot for each step.

Final Validation: Present the final, fully sorted sequence of severity codes.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>
using namespace std;

// function to swap elements
```

```
void swap(int *a, int *b) {  
    int t = *a;  
    *a = *b;  
    *b = t;  
}
```

```
// function to print the array  
void printArray(int array[], int size) {  
    int i;  
    for (i = 0; i < size; i++)  
        cout << array[i] << " ";  
    cout << endl;  
}
```

```
int partition(int array[], int low, int high) {  
  
    int pivot = array[high];  
  
    int i = (low - 1);  
  
    for (int j = low; j < high; j++) {  
        if (array[j] <= pivot) {  
  
            i++;  
  
            swap(&array[i], &array[j]);  
        }  
    }  
  
    swap(&array[i + 1], &array[high]);  
  
    return (i + 1);  
}
```

```
void quickSort(int array[], int low, int high) {  
    if (low < high) {  
        int pi = partition(array, low, high);  
        quickSort(array, low, pi - 1);  
    }
```

```

        quickSort(array, pi + 1, high);
    }
}

int main() {
    int data[] = {8, 7, 6, 1, 0, 9, 2};
    int n = sizeof(data) / sizeof(data[0]);

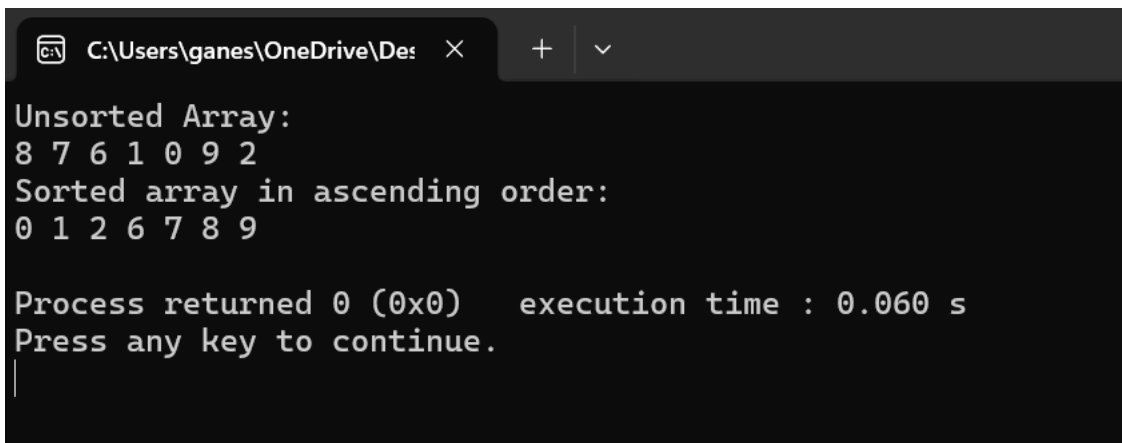
    cout << "Unsorted Array: \n";
    printArray(data, n);

    quickSort(data, 0, n - 1);

    cout << "Sorted array in ascending order: \n";
    printArray(data, n);
}

```

OUTPUT:



```

C:\Users\ganes\OneDrive\Des
Unsorted Array:
8 7 6 1 0 9 2
Sorted array in ascending order:
0 1 2 6 7 8 9

Process returned 0 (0x0)   execution time : 0.060 s
Press any key to continue.
|

```

Q5: Merge Sort

Using the Transaction IDs {38, 27, 43, 3, 9, 82, 10, 19}, implement Merge Sort to arrange the values in ascending order. Clearly illustrate:

1. The Divide Phase:
 - Show how the list is repeatedly split until single-element sub lists are obtained.
2. The Conquer (Merge) Phase:
 - Show each merge step as subsists are combined back into larger sorted lists.
3. Final Output:

Present the fully sorted list after the Merge Sort algorithm completes.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>
using namespace std;

class MergeSort {
private:

    void merge(int arr[], int left, int mid, int right) {

        int n1 = mid - left + 1;
        int n2 = right - mid;

        int L[n1], R[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j])
                arr[k++] = L[i++];
            else
                arr[k++] = R[j++];
        }

        while (i < n1)
            arr[k++] = L[i++];

        while (j < n2)
            arr[k++] = R[j++];
    }

public:
```

```

void mergeSort(int arr[], int left, int right) {
    if (left < right) {

        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

void display(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

};

int main() {

    int arr[] = {38, 27, 43, 3, 9, 82, 10, 19};
    int n = 8;

    MergeSort obj;

    cout << "Unsorted Transaction IDs: ";
    obj.display(arr, n);

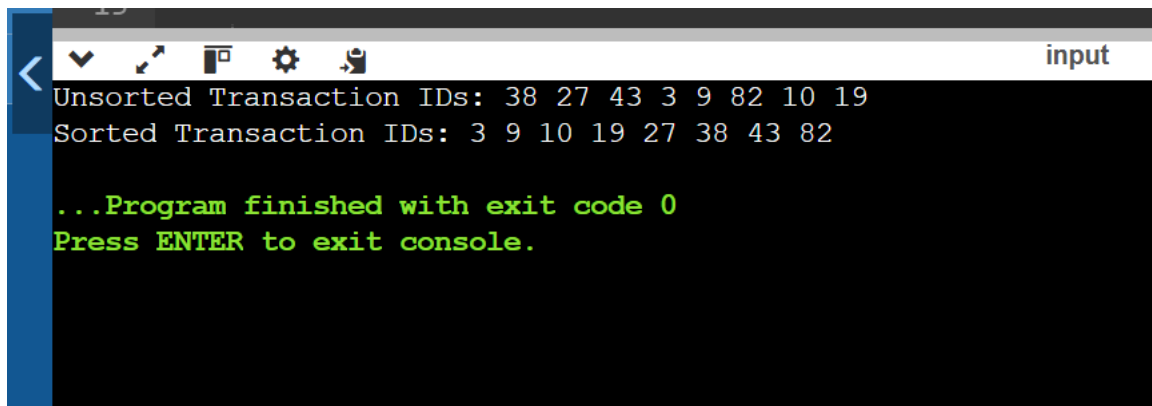
    obj.mergeSort(arr, 0, n - 1);

    cout << "\nSorted Transaction IDs: ";
    obj.display(arr, n);

    return 0;
}

```

OUTPUT:

A screenshot of a terminal window with a dark background and a blue sidebar on the left. The terminal shows the output of a program. The first line is 'Unsorted Transaction IDs: 38 27 43 3 9 82 10 19'. The second line is 'Sorted Transaction IDs: 3 9 10 19 27 38 43 82'. The third line is '...Program finished with exit code 0'. The fourth line is 'Press ENTER to exit console.' The terminal window has a title bar with the word 'input' on the right and several icons on the left.

```
< input
Unsorted Transaction IDs: 38 27 43 3 9 82 10 19
Sorted Transaction IDs: 3 9 10 19 27 38 43 82

...Program finished with exit code 0
Press ENTER to exit console.
```

Experiment: 2

Array and Linked list implementation of List, Stack and Queue ADTs.

Q1: Implement Stack using array and linked list

Case Study: The "Undo" Feature in a Text Editor

Scenario: You are a backend developer for a new minimal text editor called "QuickEdit." The product manager wants to implement a robust Undo feature. Every time a user performs an action (like typing a word or formatting text), that action is assigned a unique integer ID (e.g., Action 1, Action 2, etc.).

When the user presses "Undo" (Ctrl + Z), the editor must reverse the *most recent* action. If they press it again, it reverses the action before that. This "Last-In, First-Out" (LIFO) behavior requires a Stack data structure.

Problem Statement:

1. Store the Action IDs: 1, 2, 3, 4, 5, 6 as they occur.
2. Implement the storage mechanism using two different underlying data structures:
 - Approach A: A static Array (assume a maximum capacity of 10).
 - Approach B: A dynamic Linked List.
3. Demonstrate the PUSH operation (saving an action) and the POP operation (undoing an action).

Implement the above case study using C++/Python

CODE:

```
#include <iostream>

using namespace std;

/* ===== STACK USING ARRAY ===== */

class StackArray {
    int arr[10];
    int top;
public:
    StackArray() {
        top = -1;
    }
    void push(int x) {
        if (top == 9) return;
        arr[++top] = x;
    }
    void pop() {
        if (top == -1) return;
        cout << "Undo Action ID: " << arr[top--] << endl;
    }
};

/* ===== STACK USING LINKED LIST ===== */

class StackLL {
    struct Node {
        int data;
```

```

        Node* next;

};

Node* top;

public:

    StackLL() {
        top = NULL;
    }

    void push(int x) {
        Node* newNode = new Node();
        newNode->data = x;
        newNode->next = top;
        top = newNode;
    }

    void pop() {
        if (top == NULL) return;
        cout << "Undo Action ID: " << top->data << endl;
        Node* temp = top;
        top = top->next;
        delete temp;
    }

};

int main() {
    cout << "STACK USING ARRAY\n";
    StackArray sa;
    for (int i = 1; i <= 6; i++) sa.push(i);

```

```

    sa.pop();

    sa.pop();

    cout << "\nSTACK USING LINKED LIST\n";

    StackLL sl;

    for (int i = 1; i <= 6; i++) sl.push(i);

    sl.pop();

    sl.pop();

    return 0;

}

```

OUTPUT:

```

STACK USING ARRAY
Undo Action ID: 6
Undo Action ID: 5

STACK USING LINKED LIST
Undo Action ID: 6
Undo Action ID: 5

Process returned 0 (0x0)   execution time : 0.066 s
Press any key to continue.
|

```

Q2: Implement Queue using array and linked list

Case Study: The "Print Job" Spooler

Scenario: You are developing the driver software for a shared office printer. The printer can only print one document at a time, but multiple employees might send documents simultaneously. To handle this, you need a buffer (a Queue) that holds the Print Job IDs in the exact order they arrive.

The printer operates on a FIFO (First-In, First-Out) principle: the first file sent to the printer is the first one printed.

Problem Statement:

1. Receive Print Jobs: Store Job IDs 1, 2, 3, 4, 5, 6 as they arrive.
2. Implement the Buffer: Use two different underlying data structures:

- Approach A: A Linear Array (assume a maximum capacity of 10).
- Approach B: A Linked List (for dynamic memory allocation).

3. Perform Operations:

- Push (Enqueue): Add the incoming jobs to the queue.
- Pop (Dequeue): The printer finishes the first job and removes it from the queue.

Implement the above case study using C++/Python

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
/* ===== QUEUE USING ARRAY ===== */
```

```
class QueueArray {
```

```
    int arr[10];
```

```
    int front, rear;
```

```
public:
```

```
    QueueArray() {
```

```
        front = rear = -1;
```

```
    }
```

```
    void enqueue(int x) {
```

```
        if (rear == 9) return;
```

```
        if (front == -1) front = 0;
```

```
        arr[++rear] = x;
```

```
    }
```

```
void dequeue() {  
    if (front == -1 || front > rear) return;  
    cout << "Printing Job ID: " << arr[front++] << endl;  
}  
};
```

```
/* ===== QUEUE USING LINKED LIST ===== */
```

```
class QueueLL {  
    struct Node {  
        int data;  
        Node* next;  
    };  
    Node *front, *rear;  
  
public:  
    QueueLL() {  
        front = rear = NULL;  
    }
```

```
void enqueue(int x) {  
    Node* newNode = new Node();  
    newNode->data = x;  
    newNode->next = NULL;
```

```
    if (rear == NULL) {  
        front = rear = newNode;  
        return;  
    }  
    rear->next = newNode;  
    rear = newNode;  
}
```

```
void dequeue() {  
    if (front == NULL) return;  
    cout << "Printing Job ID: " << front->data << endl;  
    Node* temp = front;  
    front = front->next;  
    if (front == NULL) rear = NULL;  
    delete temp;  
}  
};
```

```
int main() {  
    cout << "QUEUE USING ARRAY\n";  
    QueueArray qa;  
    for (int i = 1; i <= 6; i++) qa.enqueue(i);  
    qa.dequeue();  
    qa.dequeue();  
    cout << "\nQUEUE USING LINKED LIST\n";
```

```

QueueLL ql;

for (int i = 1; i <= 6; i++) ql.enqueue(i);

ql.dequeue();

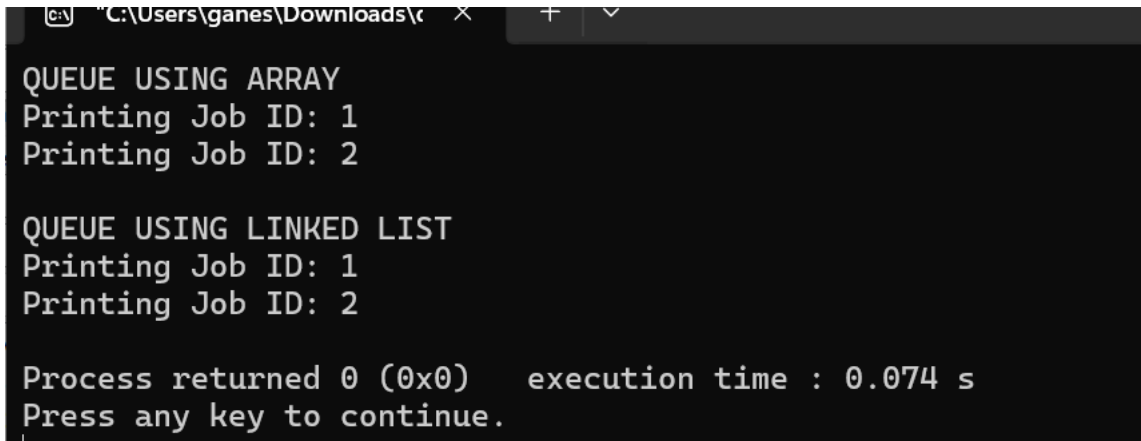
ql.dequeue();

return 0;

}

```

OUTPUT:



```

C:\Users\ganes\Downloads\c... x + v
QUEUE USING ARRAY
Printing Job ID: 1
Printing Job ID: 2

QUEUE USING LINKED LIST
Printing Job ID: 1
Printing Job ID: 2

Process returned 0 (0x0)   execution time : 0.074 s
Press any key to continue.

```

Experiment: 3

Doubly linked list and circular linked list

Q1: Doubly Linked List

Case Study: The "VibeStream" Music Playlist Engine

Scenario: You are developing the core "Now Playing" queue for a music streaming app called *VibeStream*. Unlike a basic list, this queue allows users to navigate tracks forward (next) and backward (prev).

The playlist is implemented as a Doubly Linked List where each node represents a Song.

Part 1: Insertion Scenarios

The user interacts with the app in three different ways. Your code must handle adding songs to the DLL based on these actions.

1. Insertion at Beginning (The "Play Next" Feature)

- User Action: The user finds a new song and selects "Play Immediately". This song needs to jump to the very front of the queue to be played right now, pushing the current first song to the second position.
- DLL Operation: InsertAtHead
- Logic:
 1. Create a NewNode (The new song).
 2. Set NewNode->next = Head.
 3. Set Head->prev = NewNode.
 4. Update Head = NewNode.

2. Insertion at Middle (The "Add After This Song" Feature)

- User Action: The user is looking at a specific song in the queue (let's say song #3, "Shape of You") and chooses "Add to Queue > After this song". The new song must be inserted directly between song #3 and song #4.
- DLL Operation: InsertAfterNode
- Logic:
 1. Traverse to the specific node (CurrentNode).
 2. Create NewNode.
 3. Set NewNode->next = CurrentNode->next.
 4. Set CurrentNode->next->prev = NewNode.
 5. Set CurrentNode->next = NewNode.
 6. Set NewNode->prev = CurrentNode.

3. Insertion at End (The "Add to Queue" Feature)

- User Action: The user browses an album and clicks "Add to Queue". This is the default behavior; the song is added to the very bottom of the playlist.
- DLL Operation: InsertAtTail
- Logic:
 1. Traverse to the last node (Tail).
 2. Create NewNode.
 3. Set Tail->next = NewNode.

4. Set `NewNode->prev = Tail`.
5. Set `NewNode->next = NULL` (New tail).

Part 2: Deletion Scenarios

The user decides to clean up their playlist. Your code must handle removing nodes from different positions.

1. Deletion at Beginning (Song Finished)

- User Action: The first song in the playlist finishes playing. The app automatically removes it from the top of the queue to load the next song.
- DLL Operation: `DeleteHead`
- Logic:
 1. Identify the Head node.
 2. Move Head to `Head->next` (The second song becomes the new first song).
 3. Set the new Head `->prev = NULL`.
 4. Free the memory of the old Head node.

2. Deletion at Middle (The "Remove Track" Feature)

- User Action: The user scrolls through the upcoming tracks, sees a song they dislike in the middle of the list, and swipes left to "Delete".
- DLL Operation: `DeleteSpecificNode`
- Logic:
 1. Traverse to the `TargetNode`.
 2. Set `TargetNode->prev->next = TargetNode->next` (Bypass target going forward).
 3. Set `TargetNode->next->prev = TargetNode->prev` (Bypass target going backward).
 4. Free the `TargetNode`.

3. Deletion at End (The "Undo Last Add" Feature)

- User Action: The user accidentally added the wrong song to the bottom of the queue and clicks "Undo". The last song needs to be removed.
- DLL Operation: `DeleteTail`
- Logic:

1. Traverse to the Tail node.
2. Set Tail->prev->next = NULL (The second-to-last node becomes the new tail).
3. Free the old Tail node.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>
#include <string>
using namespace std;

/* Song Node */
class Song {
public:
    string title;
    Song* prev;
    Song* next;

    Song(string t) {
        title = t;
        prev = NULL;
        next = NULL;
    }
};

/* Playlist Class */
class Playlist {
private:
    Song* head;

public:
    Playlist() {
        head = NULL;
    }

    /* ----- INSERT OPERATIONS ----- */

    // 1. Insert at Beginning (Play Immediately)
    void insertAtHead(string title) {
        Song* newNode = new Song(title);
```

```

    if (head != NULL) {
        newNode->next = head;
        head->prev = newNode;
    }
    head = newNode;
}

// 2. Insert After a Specific Song
void insertAfter(string afterTitle, string newTitle) {
    Song* current = head;

    while (current != NULL && current->title != afterTitle) {
        current = current->next;
    }

    if (current == NULL) {
        cout << "Song not found!\n";
        return;
    }

    Song* newNode = new Song(newTitle);
    newNode->next = current->next;
    newNode->prev = current;

    if (current->next != NULL) {
        current->next->prev = newNode;
    }

    current->next = newNode;
}

// 3. Insert at End (Add to Queue)
void insertAtTail(string title) {
    Song* newNode = new Song(title);

    if (head == NULL) {
        head = newNode;
        return;
    }

```

```

    Song* tail = head;
    while (tail->next != NULL) {
        tail = tail->next;
    }

    tail->next = newNode;
    newNode->prev = tail;
}

/* ----- DELETE OPERATIONS ----- */

// 1. Delete at Beginning (Song Finished)
void deleteHead() {
    if (head == NULL) return;

    Song* temp = head;
    head = head->next;

    if (head != NULL) {
        head->prev = NULL;
    }

    delete temp;
}

// 2. Delete Specific Song (Remove Track)
void deleteSong(string title) {
    Song* current = head;

    while (current != NULL && current->title != title) {
        current = current->next;
    }

    if (current == NULL) {
        cout << "Song not found!\n";
        return;
    }

    if (current == head) {
        deleteHead();
        return;
    }

```

```

    }

    if (current->next != NULL) {

        current->next->prev = current->prev;
    }

    if (current->prev != NULL) {
        current->prev->next = current->next;
    }

    delete current;
}

// 3. Delete at End (Undo Last Add)
void deleteTail() {
    if (head == NULL) return;

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Song* tail = head;
    while (tail->next != NULL) {
        tail = tail->next;
    }

    tail->prev->next = NULL;
    delete tail;
}

/* ----- DISPLAY ----- */

void displayForward() {
    Song* temp = head;
    cout << "Now Playing Queue (Forward):\n";
    while (temp != NULL) {
        cout << temp->title << " <-> ";
        temp = temp->next;
    }
}

```

```

        cout << "NULL\n";
    }

void displayBackward() {
    if (head == NULL) return;

    Song* tail = head;
    while (tail->next != NULL) {
        tail = tail->next;
    }

    cout << "Queue (Backward):\n";
    while (tail != NULL) {
        cout << tail->title << " <-> ";
        tail = tail->prev;
    }
    cout << "NULL\n";
}

};

/* ----- MAIN FUNCTION ----- */

int main() {
    Playlist vibeStream;

    vibeStream.insertAtTail("Believer");
    vibeStream.insertAtTail("Shape of You");
    vibeStream.insertAtTail("Blinding Lights");

    vibeStream.insertAtHead("Perfect");    // Play Immediately
    vibeStream.insertAfter("Shape of You", "Happier");

    vibeStream.displayForward();

    vibeStream.deleteSong("Shape of You");    // Remove Track
    vibeStream.deleteTail();    // Undo Last Add
    vibeStream.deleteHead();    // Song Finished

    vibeStream.displayForward();
    vibeStream.displayBackward();
}

```

```
    return 0;
}
```

OUTPUT:

```
C:\Users\ganes\OneDrive\Dei X + v
Now Playing Queue (Forward):
Perfect <-> Believer <-> Shape of You <-> Happier <-> Blinding Lights <-> NULL
Now Playing Queue (Forward):
Believer <-> Happier <-> NULL
Queue (Backward):
Happier <-> Believer <-> NULL

Process returned 0 (0x0)   execution time : 0.086 s
Press any key to continue.
```

Q2: Circular Linked List

Problem Statement: The Metro Loop Trace

Scenario: You are managing the navigation system for a city Metro that operates on a continuous loop (Circular Linked List). The system currently holds the following station sequence: Current State: [HEAD: Central] → ParkStreet → Museum → LakeView → (loops back to Central)

Task: Trace the state of the linked list after performing the following six operations in sequence. For each step, identify the List Order, the Last Node's Target, and the Current Head.

Step-by-Step Solution

1. Insert "Airport" at Beginning

Action: Create node Airport. Point Airport to Central. Traverse to the last node (LakeView) and update its next pointer to Airport. Update Head to Airport.

New Head: Airport

List Order: Airport → Central → ParkStreet → Museum → LakeView

Loop: LakeView connects back to Airport.

2. Insert "Harbor" at the End

Action: Create node Harbor. Point Harbor to the current Head (Airport). Traverse to the current last node (LakeView) and point it to Harbor.

New Head: Airport (Unchanged)

List Order: Airport → Central → ParkStreet → Museum → LakeView → Harbor

Loop: Harbor connects back to Airport.

3. Insert "CityMall" between Museum and LakeView

Action: Traverse to Museum. Point CityMall to Museum's next node (LakeView). Update Museum to point to CityMall.

New Head: Airport (Unchanged)

List Order: Airport → Central → ParkStreet → Museum → CityMall → LakeView → Harbor

Loop: Harbor connects back to Airport.

4. Delete Beginning (Remove "Airport")

Action: The Head is Airport. Move Head pointer to Airport->next (Central). Traverse to the last node (Harbor) and update it to point to the new Head (Central). Delete Airport.

New Head: Central

List Order: Central → ParkStreet → Museum → CityMall → LakeView → Harbor

Loop: Harbor connects back to Central.

5. Delete Middle (Remove "CityMall")

Action: Traverse to find the node before CityMall, which is Museum. Change Museum->next to skip CityMall and point directly to LakeView. Delete CityMall.

New Head: Central (Unchanged)

List Order: Central → ParkStreet → Museum → LakeView → Harbor

Loop: Harbor connects back to Central.

6. Delete End (Remove "Harbor")

Action: Traverse to the node before the last node (LakeView). Point LakeView to the Head (Central). Delete Harbor.

New Head: Central (Unchanged)

List Order: Central → ParkStreet → Museum → LakeView

Loop: LakeView connects back to Central.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Node {
```

```
public:
```

```
    string station;
```

```
    Node* next;
```

```
    Node(string s) {
```

```
        station = s;
```

```
        next = NULL;
```

```
    }
```

```
};
```

```
Node* head = NULL;
```

```
// Insert at beginning
```

```
void insertAtBeginning(string station) {
```

```
    Node* newNode = new Node(station);
```

```
    if (head == NULL) {
```

```
        head = newNode;
```

```
    newNode->next = head;
    return;
}
```

```
Node* temp = head;
while (temp->next != head)
    temp = temp->next;
```

```
newNode->next = head;
temp->next = newNode;
head = newNode;
}
```

// Insert at end

```
void insertAtEnd(string station) {
    Node* newNode = new Node(station);

    if (head == NULL) {
        head = newNode;
        newNode->next = head;
        return;
    }
```

```
Node* temp = head;
while (temp->next != head)
```

```

    temp = temp->next;

    temp->next = newNode;
    newNode->next = head;
}

// Insert after a specific station
void insertAfter(string afterStation, string station) {
    Node* temp = head;

    do {
        if (temp->station == afterStation) {
            Node* newNode = new Node(station);
            newNode->next = temp->next;
            temp->next = newNode;
            return;
        }
        temp = temp->next;
    } while (temp != head);
}

// Delete beginning
void deleteBeginning() {
    if (head == NULL)
        return;

```

```
if (head->next == head) {  
    delete head;  
    head = NULL;  
    return;  
}
```

```
Node* temp = head;  
Node* last = head;
```

```
while (last->next != head)  
    last = last->next;
```

```
head = head->next;  
last->next = head;  
delete temp;  
}
```

// Delete a specific station

```
void deleteMiddle(string station) {
```

```
    if (head == NULL)  
        return;
```

```
Node* curr = head;  
Node* prev = NULL;
```

```
do {
    if (curr->station == station) {
        if (prev != NULL) {
            prev->next = curr->next;
            delete curr;
        }
        return;
    }
    prev = curr;
    curr = curr->next;
} while (curr != head);
}
```

// Delete end

```
void deleteEnd() {
    if (head == NULL)
        return;

    if (head->next == head) {
        delete head;
        head = NULL;
        return;
    }
```

```

Node* temp = head;
Node* prev = NULL;
while (temp->next != head) {
    prev = temp;
    temp = temp->next;
}
prev->next = head;
delete temp;
}
// Display list
void display() {
    if (head == NULL)
        return;
    Node* temp = head;
    do {
        cout << temp->station << " -> ";
        temp = temp->next;
    } while (temp != head);
    cout << "(back to " << head->station << ")\n";
}
int main() {
    insertAtEnd("Central");
    insertAtEnd("ParkStreet");
    insertAtEnd("Museum");
    insertAtEnd("LakeView");
}

```

```

insertAtBeginning("Airport");

insertAtEnd("Harbor");

insertAfter("Museum", "CityMall");

deleteBeginning();

deleteMiddle("CityMall");

deleteEnd();

display();

return 0;
}

```

OUTPUT:

```

Initial List:
Central -> ParkStreet -> Museum -> LakeView -> (back to Central)

After Insert Airport at Beginning:
Airport -> Central -> ParkStreet -> Museum -> LakeView -> (back to Airport)

After Insert Harbor at End:
Airport -> Central -> ParkStreet -> Museum -> LakeView -> Harbor -> (back to Airport)

After Insert CityMall after Museum:
Airport -> Central -> ParkStreet -> Museum -> CityMall -> LakeView -> Harbor -> (back to Airport)

After Delete Beginning (Airport):
Central -> ParkStreet -> Museum -> CityMall -> LakeView -> Harbor -> (back to Central)

After Delete CityMall:
Central -> ParkStreet -> Museum -> LakeView -> Harbor -> (back to Central)

After Delete End (Harbor):
Central -> ParkStreet -> Museum -> LakeView -> (back to Central)

Process returned 0 (0x0)   execution time : 0.083 s
Press any key to continue.

```

Experiment: 4

Applications of List, Stack and Queue ADTs.

Q1: EXPRESSION CONVERSION USING STACK DATA STRUCTURE

The Scenario: The Physics Engine

Imagine you are designing the compiler for a game physics engine. The engine needs to calculate the Net Impact Force during a collision. The formula given by the physics team is:

$$(M + V) * G - F/T$$

Where:

M = Mass

V = Velocity

G = Gravity

F = Friction

T = Time

To process this efficiently, the computer must convert this human-readable Infix expression into Postfix and Prefix using stack data structure.

Generate code in Python or C++ that actually implements this conversion logic.

CODE:

```
#include <iostream>
```

```
#include <stack>
```

```
#include <string>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
// Function to check operator precedence
```

```
int precedence(char op) {
```

```
    if (op == '+' || op == '-') return 1;
```

```
    if (op == '*' || op == '/') return 2;
```

```
    if (op == '^') return 3;
```

```
    return 0;
```

```
}
```

```
// Check if character is an operator
```

```
bool isOperator(char ch) {
```

```
    return (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '^');  
}
```

```
// Infix to Postfix conversion
```

```
string infixToPostfix(string infix) {
```

```
    stack<char> st;
```

```
    string postfix = "";
```

```
    for (char ch : infix) {
```

```
        // Operand
```

```
        if (isalnum(ch)) {
```

```
            postfix += ch;
```

```
        }
```

```
        // Left parenthesis
```

```
        else if (ch == '(') {
```

```
            st.push(ch);
```

```
        }
```

```
        // Right parenthesis
```

```
        else if (ch == ')') {
```

```
            while (!st.empty() && st.top() != '(') {
```

```
                postfix += st.top();
```

```
                st.pop();
```

```
            }
```

```
            st.pop(); // remove '('
```

```
        }
```

```

// Operator
else if (isOperator(ch)) {
    while (!st.empty() && precedence(st.top()) >= precedence(ch)) {
        postfix += st.top();
        st.pop();
    }
    st.push(ch);
}

while (!st.empty()) {
    postfix += st.top();
    st.pop();
}

return postfix;
}

```

```

// Infix to Prefix conversion
string infixToPrefix(string infix) {
    reverse(infix.begin(), infix.end());

    // Swap '(' with ')'
    for (char &ch : infix) {
        if (ch == '(') ch = ')';
    }
}

```

```

        else if (ch == ')') ch = '(';
    }

    string postfix = infixToPostfix(infix);
    reverse(postfix.begin(), postfix.end());

    return postfix;
}

int main() {
    string infix;

    cout << "Enter Infix Expression: ";
    cin >> infix;

    cout << "Postfix Expression: " << infixToPostfix(infix) << endl;
    cout << "Prefix Expression : " << infixToPrefix(infix) << endl;

    return 0;
}

```

OUTPUT:

```

Enter Infix Expression: p+l+r+q-r*p+q
Postfix Expression: pl+r+q+rp*-q+
Prefix Expression : +p+l+r-q+*rpq

Process returned 0 (0x0)   execution time : 27.714 s
Press any key to continue.

```

Experiment: 5

Array based and linked structure-based implementation of Binary Tree operations

Q1. Implement Binary tree using Array and perform insertion and visiting of nodes (preorder, postorder, inorder).

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
int tree[100];
```

```
int n = 0;
```

```
// Insert into binary tree (level-wise)
```

```
void insert(int value) {
```

```
    tree[n++] = value;
```

```
}
```

```
// Inorder Traversal
```

```
void inorder(int index) {
```

```
    if (index >= n) return;
```

```
    inorder(2 * index + 1);
```

```
    cout << tree[index] << " ";
```

```
    inorder(2 * index + 2);
```

```
}
```

```
// Preorder Traversal
```

```
void preorder(int index) {
```

```
    if (index >= n) return;
    cout << tree[index] << " ";
    preorder(2 * index + 1);
    preorder(2 * index + 2);
}
```

// Postorder Traversal

```
void postorder(int index) {
    if (index >= n) return;
    postorder(2 * index + 1);
    postorder(2 * index + 2);
    cout << tree[index] << " ";
}
```

```
int main() {
    insert(10);
    insert(20);
    insert(30);
    insert(40);
    insert(50);
```

```
    cout << "Inorder: ";
    inorder(0);
```

```
    cout << "\nPreorder: ";
```

```

preorder(0);

cout << "\nPostorder: ";

postorder(0);

return 0;
}

```

OUTPUT:

```

Inorder: 40 20 50 10 30
Preorder: 10 20 40 50 30
Postorder: 40 50 20 30 10
Process returned 0 (0x0)    execution time : 0.046 s
Press any key to continue.
|

```

Q2. Implement Binary tree using linked list and perform insertion and visiting of nodes (preorder, postorder, inorder).

CODE:

```

#include <iostream>

using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int val) {

```

```
        data = val;
        left = right = NULL;
    }
};
```

```
// Insert node (simple manual insertion for demo)
```

```
Node* insert(Node* root, int val) {
    if (root == NULL)
        return new Node(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}
```

```
// Inorder Traversal
```

```
void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

```
// Preorder Traversal
```

```
void preorder(Node* root) {  
    if (root == NULL) return;  
    cout << root->data << " ";  
    preorder(root->left);  
    preorder(root->right);  
}
```

```
// Postorder Traversal
```

```
void postorder(Node* root) {  
    if (root == NULL) return;  
    postorder(root->left);  
    postorder(root->right);  
    cout << root->data << " ";  
}
```

```
int main() {
```

```
    Node* root = NULL;
```

```
    root = insert(root, 10);
```

```
    insert(root, 20);
```

```
    insert(root, 5);
```

```
    insert(root, 30);
```

```
    insert(root, 15);
```

```

cout << "Inorder: ";
inorder(root);

cout << "\nPreorder: ";
preorder(root);

cout << "\nPostorder: ";
postorder(root);

return 0;
}

```

OUTPUT:

```

Inorder: 5 10 15 20 30
Preorder: 10 5 20 15 30
Postorder: 5 15 30 20 10
Process returned 0 (0x0)    execution time : 0.066 s
Press any key to continue.
|

```

Experiment: 6

Implementation of priority queues-heaps

Q1: The Scenario: The "Cloud Task Scheduler"

You are building a Cloud Computing Task Scheduler. Servers have limited CPU and RAM, so they cannot run every user's request at once. Each task sent to the cloud has:

1. Task ID: A unique identifier.
2. Priority: An integer (e.g., 1 to 100).
3. Execution Time: How long the task takes.

The Problem: If the scheduler uses a simple queue (FIFO), a massive, low-priority "Data Backup" task might block a high-priority "Emergency Security Patch" from running.

Your Goal: Implement a Max-Heap using C++/Python to ensure that whenever a server becomes free, it always picks the task with the highest priority value.

To test your implementation of a Priority Queue (Max-Heap) for the "Cloud Task Scheduler" scenario, here is a specific set of input data. You can use these values to trace how the heap structure changes and ensure the correct task is processed first.

1. The Input Dataset

Imagine these 6 tasks are submitted to your scheduler in this exact order:

Task ID	Priority Level	Initial Arrival Order
A	15	1
B	40	2
C	30	3
D	50	4
E	10	5
F	45	6

2. The Internal Heap Structure

If you implement this as a Max-Heap (using an array/list), the elements will be rearranged to satisfy the heap property (Parent Children).

After all elements are inserted, your internal array/list should look like this: [50, 45, 30, 15, 10, 40]

The Tree Visualization:

- Root: 50 (Task D)
- Left Child of 50: 45 (Task F)
- Right Child of 50: 30 (Task C)
- Left Child of 45: 15 (Task A)
- Right Child of 45: 10 (Task E)
- Left Child of 30: 40 (Task B)

3. The Expected Output (Processing Order)

When you call pop () repeatedly until the queue is empty, the tasks must come out in descending order of priority.

Output:

1. Task D (Priority: 50)
2. Task F (Priority: 45)
3. Task B (Priority: 40)
4. Task C (Priority: 30)
5. Task A (Priority: 15)
6. Task E (Priority: 10)

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
int heap[100];
```

```
int size = 0;
```

```
// Swap
```

```
void swap(int &a, int &b) {
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

```
// Insert into heap
```

```
void insert(int value) {
```

```
    size++;
```

```
    heap[size] = value;
```

```

int i = size;

while (i > 1 && heap[i] > heap[i / 2]) {
    swap(heap[i], heap[i / 2]);
    i = i / 2;
}
}

// Delete max
int deleteMax() {
    int root = heap[1];
    heap[1] = heap[size];
    size--;

    int i = 1;

    while (2 * i <= size) {
        int largest = i;
        int left = 2 * i;
        int right = 2 * i + 1;

        if (left <= size && heap[left] > heap[largest])
            largest = left;

        if (right <= size && heap[right] > heap[largest])
            largest = right;
    }
}

```

```

        if (largest != i) {
            swap(heap[i], heap[largest]);
            i = largest;
        } else {
            break;
        }
    }

    return root;
}

```

```

// Display heap

```

```

void display() {
    for (int i = 1; i <= size; i++) {
        cout << heap[i] << " ";
    }
    cout << endl;
}

```

```

int main() {
    int n, value;

    cout << "Enter number of tasks: ";
    cin >> n;

    cout << "Enter priorities:\n";
    for (int i = 0; i < n; i++) {
        cin >> value;
    }
}

```

```

        insert(value);
    }

    cout << "\nHeap structure:\n";
    display();

    cout << "\nProcessing order (Highest Priority First):\n";
    while (size > 0) {
        cout << deleteMax() << " ";
    }

    return 0;
}

```

OUTPUT:

```

Enter number of tasks: 6
Enter priorities:
15
40
30
50
10
45

Heap structure:
50 40 45 15 10 30

Processing order (Highest Priority First):
50 45 40 30 15 10

...Program finished with exit code 0
Press ENTER to exit console.

```

Experiment: 7

List based implementation of hash tables

Q1: Open Hashing or Separate Chaining

The Scenario: The "Library Book Archive"

Imagine you are a librarian at a historic archive where each book has a unique ID number (437, 325, 175, 199, 171, 189, 127, 509). To keep the archive organized and allow for lightning-fast retrieval, you use a Separate Chaining (Open Hashing) system.

You have 10 physical shelves (numbered 0 to 9). To decide which shelf a book belongs to, you use the last digit of its ID—this is your hash function: $H(X) = X \bmod 10$.

The Problem: Since many books might have the same last digit (e.g., 325 and 175), you can't just put one book on a shelf. Instead, each shelf contains a **linked list** of books. When a reader asks for a book, you go straight to the correct shelf and look through only that specific list.

1. Input Data

The following sequence is inserted into the hash table one by one:

Sequence: {437, 325, 175, 199, 171, 189, 127, 509}

2. The Internal Hashing Logic

The system calculates the index using $H(X) = X \bmod 10$. When two numbers result in the same index, the new number is appended to the list at that index (**Separate Chaining**).

- **437:** $437 \bmod 10 = 7$. (Goes to Index 7)
- **325:** $325 \bmod 10 = 5$. (Goes to Index 5)
- **175:** $175 \bmod 10 = 5$. (**Collision:** Appends to 325 at Index 5)
- **199:** $199 \bmod 10 = 9$. (Goes to Index 9)
- **171:** $171 \bmod 10 = 1$. (Goes to Index 1)
- **189:** $189 \bmod 10 = 9$. (**Collision:** Appends to 199 at Index 9)
- **127:** $127 \bmod 10 = 7$. (**Collision:** Appends to 437 at Index 7)
- **509:** $509 \bmod 10 = 9$. (**Collision:** Appends to 189 at Index 9)

3. Expected Output

If you print the hash table, the output should represent the "buckets" and their respective chains.

Index	Chain (Linked List)

0	NULL
1	171 -> NULL
2	NULL
3	NULL
4	NULL
5	325 -> 175 -> NULL
6	NULL
7	437 -> 127 -> NULL
8	NULL
9	199 -> 189 -> 509 -> NULL

4. Verification (Search Example)

If you were to search for a value, the "Expected Output" for a search function would look like this:

- **Search(175):** 1. Calculate $175 \pmod{10} = 5$.
2. Look at Index 5.
3. Traverse list: 325 (no), 175 (yes!).
4. **Result:** Found at Index 5.
- **Search(100):** 1. Calculate $100 \pmod{10} = 0$.
2. Look at Index 0.
3. Index 0 is empty.
4. **Result:** Not Found.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>

using namespace std;

class Node {
public:
```

```

int data;

Node* next;

Node(int val) {
    data = val;
    next = NULL;
}

};

Node* table[10]; // 10 buckets

// Hash Function
int hashFunction(int key) {
    return key % 10;
}

// Insert into hash table
void insert(int key) {
    int index = hashFunction(key);

    Node* newNode = new Node(key);

    if (table[index] == NULL) {
        table[index] = newNode;
    } else {
        Node* temp = table[index];
        while (temp->next != NULL) {
            temp = temp->next;

```

```
    }  
    temp->next = newNode;  
}  
}
```

```
// Display hash table
```

```
void display() {  
    for (int i = 0; i < 10; i++) {  
        cout << i << " -> ";  
  
        Node* temp = table[i];  
  
        if (temp == NULL) {  
            cout << "NULL";  
        } else {  
            while (temp != NULL) {  
                cout << temp->data << " -> ";  
                temp = temp->next;  
            }  
            cout << "NULL";  
        }  
  
        cout << endl;  
    }  
}
```

```
int main() {  
    int n, value;
```

```
// Initialize table
for (int i = 0; i < 10; i++) {
    table[i] = NULL;
}

cout << "Enter number of books: ";
cin >> n;

cout << "Enter book IDs:\n";
for (int i = 0; i < n; i++) {
    cin >> value;
    insert(value);
}

cout << "\nHash Table (Separate Chaining):\n";
display();

return 0;
}
```

OUTPUT:

```
Enter number of books: 8
Enter book IDs:
437
325
175
199
171
189
127
509

Hash Table (Separate Chaining):
0 -> NULL
1 -> 171 -> NULL
2 -> NULL
3 -> NULL
4 -> NULL
5 -> 325 -> 175 -> NULL
6 -> NULL
7 -> 437 -> 127 -> NULL
8 -> NULL
9 -> 199 -> 189 -> 509 -> NULL

...Program finished with exit code 0
Press ENTER to exit console.
```

Experiment: 8

Implementation of skip lists

Q1: SKIP LIST (Insertion and Searching)

The Scenario: The "Express Train" Transit System

Imagine you are designing a high-speed rail network for a massive metropolitan area. You have several stations (nodes) with IDs: **2, 4, 6, 7, 9, 13, 15, 16, 17**.

A standard subway (Level 0) stops at **every single station**, which is slow if you want to get from one end of the city to the other. To fix this, you build **Express Tracks** (Levels 1, 2, etc.) that skip over several stations, allowing commuters to "jump" large distances before switching back to the local track to reach their specific destination.

Your Task: Using the data points provided, implement a Skip List using C++/Python where:

1. **Insert:** Each ID is added to the base level. Then, use a "coin flip" (randomization) to decide if that station gets an Express platform on Level 1, Level 2, etc.
2. **Search:** To find a specific station (e.g., **13**), start at the highest Express level and move forward until you would overshoot the target, then drop down a level and repeat.

CODE:

```
#include <iostream>

#include <cstdlib>

using namespace std;

#define MAX_LEVEL 3

class Node {
public:
    int value;

    Node* forward[MAX_LEVEL + 1];

    Node(int val) {
        value = val;
        for (int i = 0; i <= MAX_LEVEL; i++)
            forward[i] = NULL;
    }
};

class SkipList {
    int level;

    Node* header;

public:
    SkipList() {
        level = 0;

        header = new Node(-1); // dummy node
    }
```

```
// Random level (coin flip)
```

```
int randomLevel() {  
    int lvl = 0;  
    while (rand() % 2 && lvl < MAX_LEVEL)  
        lvl++;  
    return lvl;  
}
```

```
// Insert
```

```
void insert(int value) {  
    Node* update[MAX_LEVEL + 1];  
    Node* temp = header;  
  
    for (int i = level; i >= 0; i--) {  
        while (temp->forward[i] != NULL &&  
            temp->forward[i]->value < value) {  
            temp = temp->forward[i];  
        }  
        update[i] = temp;  
    }  
}
```

```
temp = temp->forward[0];
```

```
if (temp == NULL || temp->value != value) {  
    int newLevel = randomLevel();
```

```
    if (newLevel > level) {
```

```

        for (int i = level + 1; i <= newLevel; i++)
            update[i] = header;
        level = newLevel;
    }

    Node* newNode = new Node(value);

    for (int i = 0; i <= newLevel; i++) {
        newNode->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = newNode;
    }
}
}

```

// Search

```

void search(int value) {
    Node* temp = header;

    for (int i = level; i >= 0; i--) {
        while (temp->forward[i] != NULL &&
            temp->forward[i]->value < value) {
            temp = temp->forward[i];
        }
    }

    temp = temp->forward[0];

    if (temp != NULL && temp->value == value)

```

```

        cout << "Found " << value << endl;
    else
        cout << value << " Not Found\n";
}

// Display
void display() {
    for (int i = level; i >= 0; i--) {
        Node* temp = header->forward[i];
        cout << "Level " << i << ": ";
        while (temp != NULL) {
            cout << temp->value << " ";
            temp = temp->forward[i];
        }
        cout << endl;
    }
}
};

```

```

int main() {
    SkipList list;

    int n, value;

    cout << "Enter number of stations: ";
    cin >> n;

    cout << "Enter station IDs:\n";

```

```

for (int i = 0; i < n; i++) {
    cin >> value;
    list.insert(value);
}

cout << "\nSkip List Levels:\n";
list.display();

int key;
cout << "\nEnter value to search: ";
cin >> key;

list.search(key);

return 0;
}

```

OUTPUT:

```

Enter number of stations: 9
Enter station IDs:
2
4
6
7
9
13
15
16
17

Skip List Levels:
Level 3: 4
Level 2: 4 9 15
Level 1: 2 4 9 13 15
Level 0: 2 4 6 7 9 13 15 16 17

Enter value to search: 13
Found 13

...Program finished with exit code 0
Press ENTER to exit console.

```

Q2: SKIP LIST (Deletion)

The Scenario: The "Station Closure" (use the data given in question 1)

Following our **Express Train** analogy, imagine that the city decides to close a station due to maintenance or budget cuts. Let's say station **9** needs to be removed from the network.

When a station is demolished, you can't just remove the platform on the local track (Level 0). You must also carefully remove any **Express platforms** that station had on higher levels. If you don't, the express trains will try to stop at a station that no longer exists, causing a system crash!

Your Task:

1. **Locate:** Use the same "jump and drop" search logic to find station **9**.
2. **Unlink:** At every level where station **9** exists, update the "previous" station's pointer to skip over **9** and point directly to the next available station (e.g., station **13**).
3. **Cleanup:** Free up the memory/resources used by that station.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>

#include <cstdlib>

using namespace std;

#define MAX_LEVEL 3

class Node {
public:
    int value;

    Node* forward[MAX_LEVEL + 1];

    Node(int val) {
        value = val;

        for (int i = 0; i <= MAX_LEVEL; i++)
```

```
        forward[i] = NULL;
    }
};
```

```
class SkipList {
    int level;
    Node* header;
```

```
public:
```

```
    SkipList() {
        level = 0;
        header = new Node(-1);
    }
```

```
    int randomLevel() {
        int lvl = 0;
        while (rand() % 2 && lvl < MAX_LEVEL)
            lvl++;
        return lvl;
    }
```

```
// INSERT
```

```
void insert(int value) {
    Node* update[MAX_LEVEL + 1];
    Node* temp = header;

    for (int i = level; i >= 0; i--) {
        while (temp->forward[i] != NULL &&
```

```

        temp->forward[i]->value < value) {
            temp = temp->forward[i];
        }
        update[i] = temp;
    }

    temp = temp->forward[0];

    if (temp == NULL || temp->value != value) {
        int newLevel = randomLevel();

        if (newLevel > level) {
            for (int i = level + 1; i <= newLevel; i++)
                update[i] = header;
            level = newLevel;
        }

        Node* newNode = new Node(value);

        for (int i = 0; i <= newLevel; i++) {
            newNode->forward[i] = update[i]->forward[i];
            update[i]->forward[i] = newNode;
        }
    }
}

// DELETE
void deleteValue(int value) {

```

```
Node* update[MAX_LEVEL + 1];
```

```
Node* temp = header;
```

```
// Step 1: Locate
```

```
for (int i = level; i >= 0; i--) {  
    while (temp->forward[i] != NULL &&  
           temp->forward[i]->value < value) {  
        temp = temp->forward[i];  
    }  
    update[i] = temp;  
}
```

```
temp = temp->forward[0];
```

```
// Step 2: If found → unlink
```

```
if (temp != NULL && temp->value == value) {  
    for (int i = 0; i <= level; i++) {  
        if (update[i]->forward[i] != temp)  
            break;  
  
        update[i]->forward[i] = temp->forward[i];  
    }  
}
```

```
// Step 3: Cleanup
```

```
delete temp;
```

```
// Adjust level if needed
```

```
while (level > 0 && header->forward[level] == NULL) {
```

```

        level--;
    }

    cout << value << " deleted successfully\n";
} else {
    cout << value << " not found\n";
}
}

// DISPLAY
void display() {
    for (int i = level; i >= 0; i--) {
        Node* temp = header->forward[i];
        cout << "Level " << i << ": ";
        while (temp != NULL) {
            cout << temp->value << " ";
            temp = temp->forward[i];
        }
        cout << endl;
    }
}

};

int main() {
    SkipList list;

    int n, value;

```

```
cout << "Enter number of stations: ";  
cin >> n;
```

```
cout << "Enter station IDs:\n";  
for (int i = 0; i < n; i++) {  
    cin >> value;  
    list.insert(value);  
}
```

```
cout << "\nBefore Deletion:\n";  
list.display();
```

```
int del;  
cout << "\nEnter station to delete: ";  
cin >> del;
```

```
list.deleteValue(del);
```

```
cout << "\nAfter Deletion:\n";  
list.display();
```

```
return 0;  
}
```

OUTPUT:

```
Enter number of stations: 9
Enter station IDs:
2
4
6
7
9
13
15
16
17

Before Deletion:
Level 3: 4
Level 2: 4 9 15
Level 1: 2 4 9 13 15
Level 0: 2 4 6 7 9 13 15 16 17

Enter station to delete: 9
9 deleted successfully

After Deletion:
Level 3: 4
Level 2: 4 15
Level 1: 2 4 13 15
Level 0: 2 4 6 7 13 15 16 17
```

Experiment: 9

Implementation of binary search trees, AVL trees and Splay trees

Q1: Binary Search Tree

The Scenario: "The Galactic Library Index"

You are the lead archivist for the Intergalactic Library. Because the library is constantly receiving new transmissions and occasionally losing old scrolls to black holes, you need a system that keeps the titles sorted by their **Unique Archive ID (UAID)** for fast retrieval.

Implement a BST system in **Python** or **C++** that manages the following events:

1. **Arrivals (Insertion):** The library receives books with UAIDs: 50, 30, 70, 20, 40, 60, 80.
2. **The Solar Flare (Deletion - Case 1):** UAID 20 is vaporized. Remove this leaf node.
3. **The Data Corruption (Deletion - Case 2):** UAID 30 becomes unreadable. This node has one child (40). Remove it and ensure the structure remains intact.

4. **The Policy Shift (Deletion - Case 3):** The library decides to archive the current root, UAID 50. Since this node has two children, you must replace it with its in-order successor.

Requirements:

- Create a Node class/struct.
- Write an insert function.
- Write a delete function that handles all three cases.

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
// Node structure
```

```
class Node {
```

```
public:
```

```
    int data;
```

```
    Node* left;
```

```
    Node* right;
```

```
    Node(int val) {
```

```
        data = val;
```

```
        left = right = NULL;
```

```
    }
```

```
};
```

```
// Insert
```

```
Node* insert(Node* root, int val) {
```

```
    if (root == NULL)
```

```
        return new Node(val);
```

```

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}

// Find minimum (in-order successor)
Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

// Delete (handles all 3 cases)
Node* deleteNode(Node* root, int key) {
    if (root == NULL)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);

    else if (key > root->data)
        root->right = deleteNode(root->right, key);

    else {

```

```

// Case 1: No child
if (root->left == NULL && root->right == NULL) {
    delete root;
    return NULL;
}

// Case 2: One child
else if (root->left == NULL) {
    Node* temp = root->right;
    delete root;
    return temp;
}
else if (root->right == NULL) {
    Node* temp = root->left;
    delete root;
    return temp;
}

// Case 3: Two children
Node* temp = findMin(root->right);
root->data = temp->data;
root->right = deleteNode(root->right, temp->data);
}

return root;
}

// Inorder traversal

```

```
void inorder(Node* root) {  
    if (root != NULL) {  
        inorder(root->left);  
        cout << root->data << " ";  
        inorder(root->right);  
    }  
}
```

```
int main() {  
    Node* root = NULL;  
  
    // Insert given UAIDs  
    int values[] = {50, 30, 70, 20, 40, 60, 80};  
  
    for (int i = 0; i < 7; i++)  
        root = insert(root, values[i]);  
  
    cout << "Initial BST (Inorder): ";  
    inorder(root);  
  
    // Case 1: Delete leaf (20)  
    root = deleteNode(root, 20);  
    cout << "\nAfter deleting 20 (Leaf): ";  
    inorder(root);  
  
    // Case 2: Delete node with one child (30)  
    root = deleteNode(root, 30);  
    cout << "\nAfter deleting 30 (One Child): ";
```

```

inorder(root);

// Case 3: Delete node with two children (50)
root = deleteNode(root, 50);
cout << "\nAfter deleting 50 (Two Children): ";
inorder(root);

return 0;
}

```

OUTPUT:

```

Initial BST (Inorder): 20 30 40 50 60 70 80
After deleting 20 (Leaf): 30 40 50 60 70 80
After deleting 30 (One Child): 40 50 60 70 80
After deleting 50 (Two Children): 40 60 70 80

...Program finished with exit code 0
Press ENTER to exit console.

```

Q2: AVL Trees

The Scenario: "Eco-Stream Data Optimizer"

You are designing a Carbon Footprint Monitoring System for a smart campus (like Amrita). Sensors across the campus report energy consumption levels at irregular intervals. To ensure the system can query and update these levels in logarithmic time, you decide to store the sensor data in an AVL Tree.

The following energy readings (in kW) are received by the central server in this specific order: 14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 20

Q1: Using C++/Python construct the AVL Tree by inserting the readings one by one. For every insertion that causes an imbalance, identify the type of rotation (LL, RR, LR, or RL) and show the tree's state after rebalancing.

Q2: From the final AVL Tree constructed, delete the following elements 8,7,11,14,17

CODE:

```
#include <iostream>

using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;
    int height;

    Node(int val) {
        data = val;
        left = right = NULL;
        height = 1;
    }
};

// Height
int getHeight(Node* n) {
    if (n == NULL) return 0;
    return n->height;
}

// Balance factor
int getBalance(Node* n) {
    if (n == NULL) return 0;
    return getHeight(n->left) - getHeight(n->right);
}
```

```
}
```

```
// Right Rotate (LL)
```

```
Node* rightRotate(Node* y) {
```

```
    Node* x = y->left;
```

```
    Node* T2 = x->right;
```

```
    x->right = y;
```

```
    y->left = T2;
```

```
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
```

```
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
```

```
    return x;
```

```
}
```

```
// Left Rotate (RR)
```

```
Node* leftRotate(Node* x) {
```

```
    Node* y = x->right;
```

```
    Node* T2 = y->left;
```

```
    y->left = x;
```

```
    x->right = T2;
```

```
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
```

```
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
```

```
    return y;
```

```
}
```

```
// Insert
```

```
Node* insert(Node* root, int key) {
```

```
    if (root == NULL)
```

```
        return new Node(key);
```

```
    if (key < root->data)
```

```
        root->left = insert(root->left, key);
```

```
    else
```

```
        root->right = insert(root->right, key);
```

```
    root->height = 1 + max(getHeight(root->left), getHeight(root->right));
```

```
    int balance = getBalance(root);
```

```
    // LL
```

```
    if (balance > 1 && key < root->left->data)
```

```
        return rightRotate(root);
```

```
    // RR
```

```
    if (balance < -1 && key > root->right->data)
```

```
        return leftRotate(root);
```

```
    // LR
```

```
    if (balance > 1 && key > root->left->data) {
```

```
        root->left = leftRotate(root->left);
```

```
        return rightRotate(root);
```

```
}
```

```
// RL
```

```
if (balance < -1 && key < root->right->data) {  
    root->right = rightRotate(root->right);  
    return leftRotate(root);  
}
```

```
return root;
```

```
}
```

```
// Find min
```

```
Node* findMin(Node* root) {  
    while (root->left != NULL)  
        root = root->left;  
    return root;  
}
```

```
// Delete
```

```
Node* deleteNode(Node* root, int key) {  
    if (root == NULL) return root;  
  
    if (key < root->data)  
        root->left = deleteNode(root->left, key);  
    else if (key > root->data)  
        root->right = deleteNode(root->right, key);  
    else {  
        if (root->left == NULL || root->right == NULL) {
```

```

Node* temp = root->left ? root->left : root->right;

if (temp == NULL) {
    temp = root;
    root = NULL;
} else {
    *root = *temp;
}

delete temp;
} else {
    Node* temp = findMin(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
}

if (root == NULL) return root;

root->height = 1 + max(getHeight(root->left), getHeight(root->right));
int balance = getBalance(root);

// LL
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);

// LR
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);

```

```

        return rightRotate(root);
    }

    // RR
    if (balance < -1 && getBalance(root->right) <= 0)
        return leftRotate(root);

    // RL
    if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}

// Inorder
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

int main() {
    Node* root = NULL;

```

```
int arr[] = {14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 20};
```

```
cout << "Insertion Process:\n";
```

```
for (int i = 0; i < 13; i++) {
```

```
    root = insert(root, arr[i]);
```

```
    cout << "After inserting " << arr[i] << ": ";
```

```
    inorder(root);
```

```
    cout << endl;
```

```
}
```

```
cout << "\nDeletion Process:\n";
```

```
int del[] = {8, 7, 11, 14, 17};
```

```
for (int i = 0; i < 5; i++) {
```

```
    root = deleteNode(root, del[i]);
```

```
    cout << "After deleting " << del[i] << ": ";
```

```
    inorder(root);
```

```
    cout << endl;
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:

```
Insertion Process:
After inserting 14: 14
After inserting 17: 14 17
After inserting 11: 11 14 17
After inserting 7: 7 11 14 17
After inserting 53: 7 11 14 17 53
After inserting 4: 4 7 11 14 17 53
After inserting 13: 4 7 11 13 14 17 53
After inserting 12: 4 7 11 12 13 14 17 53
After inserting 8: 4 7 8 11 12 13 14 17 53
After inserting 60: 4 7 8 11 12 13 14 17 53 60
After inserting 19: 4 7 8 11 12 13 14 17 19 53 60
After inserting 16: 4 7 8 11 12 13 14 16 17 19 53 60
After inserting 20: 4 7 8 11 12 13 14 16 17 19 20 53 60

Deletion Process:
After deleting 8: 4 7 11 12 13 14 16 17 19 20 53 60
After deleting 7: 4 11 12 13 14 16 17 19 20 53 60
After deleting 11: 4 12 13 14 16 17 19 20 53 60
After deleting 14: 4 12 13 16 17 19 20 53 60
After deleting 17: 4 12 13 16 19 20 53 60

...Program finished with exit code 0
Press ENTER to exit console.
```

Q3: Splay Trees

1: SPLAY TREE (Insertion and Searching)

The Scenario: The "Recently Accessed" Cache

Imagine you are building a Smart Search Bar for a digital library. The library has thousands of books, but users tend to search for the same few books repeatedly within a short period.

To optimize this, you decide to use a Splay Tree. Unlike a regular Binary Search Tree (BST), a Splay Tree performs a "Splaying" operation every time a node is accessed (searched or inserted). This operation moves that specific node to the root of the tree using a series of rotations.

Your Task: Insert the following book IDs in order: 15, 10, 17, 7, 13, 16. After each insertion, the newly added ID must become the root. Then, perform a search for ID **7** and observe how the tree structure shifts to bring **7** to the top for faster future access.

Implement the above scenario using C++/Python

CODE:

```
#include <iostream>

using namespace std;

class Node {
public:
    int key;
    Node *left, *right;

    Node(int k) {
        key = k;
        left = right = NULL;
    }
};

// Right Rotate
Node* rightRotate(Node* x) {
    Node* y = x->left;
    x->left = y->right;
    y->right = x;
    return y;
}

// Left Rotate
Node* leftRotate(Node* x) {
    Node* y = x->right;
    x->right = y->left;
```

```

y->left = x;

return y;
}

// Splay
Node* splay(Node* root, int key) {
    if (root == NULL || root->key == key)
        return root;

    // LEFT
    if (key < root->key) {
        if (root->left == NULL) return root;

        if (key < root->left->key) {
            root->left->left = splay(root->left->left, key);
            root = rightRotate(root);
        }
        else if (key > root->left->key) {
            root->left->right = splay(root->left->right, key);
            if (root->left->right != NULL)
                root->left = leftRotate(root->left);
        }

        return (root->left == NULL) ? root : rightRotate(root);
    }

    // RIGHT
    else {

```

```

if (root->right == NULL) return root;

if (key > root->right->key) {
    root->right->right = splay(root->right->right, key);
    root = leftRotate(root);
}
else if (key < root->right->key) {
    root->right->left = splay(root->right->left, key);
    if (root->right->left != NULL)
        root->right = rightRotate(root->right);
}

return (root->right == NULL) ? root : leftRotate(root);
}
}

```

// Insert

```

Node* insert(Node* root, int key) {
    if (root == NULL)
        return new Node(key);

    root = splay(root, key);

    if (root->key == key)
        return root;

    Node* newNode = new Node(key);

```

```
if (key < root->key) {
    newNode->right = root;
    newNode->left = root->left;
    root->left = NULL;
} else {
    newNode->left = root;
    newNode->right = root->right;
    root->right = NULL;
}

return newNode;
}
```

```
// Search
Node* search(Node* root, int key) {
    return splay(root, key);
}
```

```
// Inorder
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->key << " ";
        inorder(root->right);
    }
}
```

```
int main() {
```

```
Node* root = NULL;
```

```
int n, value;
```

```
cout << "Enter number of elements: ";
```

```
cin >> n;
```

```
cout << "Enter elements:\n";
```

```
for (int i = 0; i < n; i++) {
```

```
    cin >> value;
```

```
    root = insert(root, value);
```

```
    cout << "After inserting " << value << " (Root = " << root->key << "): ";
```

```
    inorder(root);
```

```
    cout << endl;
```

```
}
```

```
int key;
```

```
cout << "\nEnter value to search: ";
```

```
cin >> key;
```

```
root = search(root, key);
```

```
cout << "After searching " << key << " (Root = " << root->key << "): ";
```

```
inorder(root);
```

```
return 0;
```

```
}
```

OUTPUT:

```

Enter number of elements: 6
Enter elements:
15
After inserting 15 (Root = 15): 15
10
After inserting 10 (Root = 10): 10 15
17
After inserting 17 (Root = 17): 10 15 17
7
After inserting 7 (Root = 7): 7 10 15 17
13
After inserting 13 (Root = 13): 7 10 13 15 17
16
After inserting 16 (Root = 16): 7 10 13 15 16 17

Enter value to search: 7
After searching 7 (Root = 7): 7 10 13 15 16 17

...Program finished with exit code 0
Press ENTER to exit console.

```

2: SPLAY TREE (Deletion)

The Scenario: The "Expiring Session" Cache

Imagine you are managing a Web Session Cache for a high-traffic e-commerce site. Each node in your Splay Tree represents an active User Session ID.

Because it's a Splay Tree, the most recently active users (those who just logged in or clicked a link) are moved to the root for near-instant access. However, when a user logs out or their session expires after inactivity, you must delete their ID from the tree.

Your Task: Using the tree with IDs [15, 10, 17, 7, 13, 16], perform the following:

1. Splay the target ID (e.g., 13) to the root.
2. Delete the root.

Implement the above scenario C++/Python

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
class Node {
```

```
public:
```

```
    int key;
```

```
Node *left, *right;
```

```
Node(int k) {  
    key = k;  
    left = right = NULL;  
}  
};
```

```
// Right Rotate
```

```
Node* rightRotate(Node* x) {  
    Node* y = x->left;  
    x->left = y->right;  
    y->right = x;  
    return y;  
}
```

```
// Left Rotate
```

```
Node* leftRotate(Node* x) {  
    Node* y = x->right;  
    x->right = y->left;  
    y->left = x;  
    return y;  
}
```

```
// Splay
```

```
Node* splay(Node* root, int key) {  
    if (root == NULL || root->key == key)  
        return root;
```

```

// LEFT
if (key < root->key) {
    if (root->left == NULL) return root;

    if (key < root->left->key) {
        root->left->left = splay(root->left->left, key);
        root = rightRotate(root);
    }
    else if (key > root->left->key) {
        root->left->right = splay(root->left->right, key);
        if (root->left->right != NULL)
            root->left = leftRotate(root->left);
    }

    return (root->left == NULL) ? root : rightRotate(root);
}

```

```

// RIGHT
else {
    if (root->right == NULL) return root;

    if (key > root->right->key) {
        root->right->right = splay(root->right->right, key);
        root = leftRotate(root);
    }
    else if (key < root->right->key) {
        root->right->left = splay(root->right->left, key);
    }
}

```

```

        if (root->right->left != NULL)
            root->right = rightRotate(root->right);
    }

    return (root->right == NULL) ? root : leftRotate(root);
}
}

```

// Insert

```

Node* insert(Node* root, int key) {
    if (root == NULL)
        return new Node(key);

    root = splay(root, key);

    if (root->key == key)
        return root;

    Node* newNode = new Node(key);

    if (key < root->key) {
        newNode->right = root;
        newNode->left = root->left;
        root->left = NULL;
    } else {
        newNode->left = root;
        newNode->right = root->right;
        root->right = NULL;
    }
}

```

```
}
```

```
return newNode;
```

```
}
```

```
// Delete
```

```
Node* deleteNode(Node* root, int key) {
```

```
    if (root == NULL) return NULL;
```

```
    // Step 1: Splay target to root
```

```
    root = splay(root, key);
```

```
    // If not found
```

```
    if (root->key != key) {
```

```
        cout << "Element not found\n";
```

```
        return root;
```

```
    }
```

```
    Node* temp;
```

```
    // Step 2: Delete root
```

```
    if (root->left == NULL) {
```

```
        temp = root->right;
```

```
    } else {
```

```
        temp = root->left;
```

```
    // Bring max of left subtree to root
```

```
    temp = splay(temp, key);
```

```
temp->right = root->right;  
}
```

```
delete root;  
return temp;  
}
```

```
// Inorder
```

```
void inorder(Node* root) {  
    if (root != NULL) {  
        inorder(root->left);  
        cout << root->key << " ";  
        inorder(root->right);  
    }  
}
```

```
int main() {  
    Node* root = NULL;
```

```
    int n, value;
```

```
    cout << "Enter number of elements: ";  
    cin >> n;
```

```
    cout << "Enter elements:\n";  
    for (int i = 0; i < n; i++) {  
        cin >> value;
```

```

        root = insert(root, value);
    }

    cout << "\nInitial Tree (Inorder): ";
    inorder(root);

    int del;

    cout << "\nEnter element to delete: ";
    cin >> del;

    root = deleteNode(root, del);

    cout << "\nAfter Deletion (Inorder): ";
    inorder(root);

    return 0;
}

```

OUTPUT:

```

Enter number of elements: 6
Enter elements:
15
10
7
18
13
16

Initial Tree (Inorder): 7 10 13 15 16 18
Enter element to delete: 13

After Deletion (Inorder): 7 10 15 16 18

...Program finished with exit code 0
Press ENTER to exit console.

```

Experiment: 10

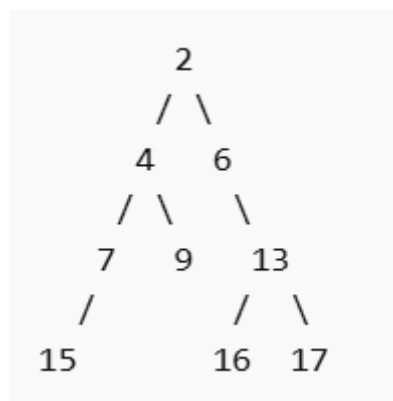
Implementation of graph traversals using BFS and DFS

The Scenario: The "Viral Post" Analysis

Imagine you are a data scientist at a social media company. A user, **Node 2**, has just posted a viral video. You want to track how this video spreads through a small cluster of friends.

The friendship connections (the graph) are represented by the IDs from your previous data:

- **2** is friends with **4** and **6**.
- **4** is friends with **7** and **9**.
- **6** is friends with **13**.
- **7** is friends with **15**.
- **13** is friends with **16** and **17**.



Your Tasks:

1. **BFS Task:** Find the "Degrees of Separation." Who sees the post first (direct friends), who sees it second (friends of friends), and so on?
2. **DFS Task:** Follow a "Rabbit Hole." Pick one friend and follow their specific connection chain as deep as possible before backtracking to the next person.

Implement BFS and DFS using C++/ Python

CODE:

```
#include <iostream>

#include <vector>

#include <queue>

using namespace std;
```

```
vector<int> adj[100];  
  
bool visited[100];  
  
// BFS  
void bfs(int start) {  
    queue<int> q;  
    q.push(start);  
    visited[start] = true;  
  
    cout << "BFS Traversal: ";  
  
    while (!q.empty()) {  
        int node = q.front();  
        q.pop();  
  
        cout << node << " ";  
  
        for (int i = 0; i < adj[node].size(); i++) {  
            int neighbor = adj[node][i];  
            if (!visited[neighbor]) {  
                visited[neighbor] = true;  
                q.push(neighbor);  
            }  
        }  
    }  
}
```

```
// DFS

void dfs(int node) {
    visited[node] = true;
    cout << node << " ";

    for (int i = 0; i < adj[node].size(); i++) {
        int neighbor = adj[node][i];
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }
}
```

```
int main() {
    int n, edges;

    cout << "Enter number of nodes: ";
    cin >> n;

    cout << "Enter number of edges: ";
    cin >> edges;

    cout << "Enter edges (u v):\n";
    for (int i = 0; i < edges; i++) {
        int u, v;
        cin >> u >> v;

        adj[u].push_back(v);
```

```

        adj[v].push_back(u); // undirected graph
    }

    int start;

    cout << "Enter starting node: ";
    cin >> start;

    // BFS
    for (int i = 0; i < 100; i++) visited[i] = false;
    bfs(start);

    // DFS
    for (int i = 0; i < 100; i++) visited[i] = false;
    cout << "\nDFS Traversal: ";
    dfs(start);

    return 0;
}

```

OUTPUT:

```

Enter number of nodes: 9
Enter number of edges: 7
Enter edges (u v):
2 4
2 6
4 7
4 9
6 13
7 15
13 16
Enter starting node: 2
BFS Traversal: 2 4 6 7 9 13 15 16
DFS Traversal: 2 4 7 15 9 6 13 16

...Program finished with exit code 0
Press ENTER to exit console.

```